Parameterized Quantum Circuits and Variational Objectives

EVA: Quantum Machine Learning – Lecture 6

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

Roadmap

1. From feature maps to trainable circuits
2. Expectation values as objectives
3. Variational Quantum Eigensolver (VQE) as prototype
4. Optimizers, noise, and practical workflow

From circuits to trainable models

Bridge from Lecture 5

Lecture 5 used fixed feature maps to create state geometry for kernels.

Lecture 6 keeps the same state-preparation view and adds trainable parameters:

$$|\psi(x,\theta)\rangle = U_{\text{ansatz}}(\theta)\,U_{\text{enc}}(x)\,|0\rangle^{\otimes n}\,.$$

The central question is now: how do we choose θ to minimize a task objective?

Parameterized quantum circuit (PQC)

A parameterized quantum circuit has tunable gates:

$$|\psi(\theta)\rangle = U(\theta)\,|0\rangle^{\otimes n}\,.$$

Typical hybrid separation:

- **encoding block** depends on data x
- **ansatz block** depends on trainable parameters θ

Reminder: every qubit starts in $|0\rangle$ by default unless we prepare a different initial state (for example, applying X to get $|1\rangle$).

From architecture to objective

Ansatz choices and trade-offs

- Hardware-efficient ansatz: shallow, device-friendly, often easier to run
- Problem-inspired ansatz: uses structure of the target task or Hamiltonian

Practical trade-off:

- Too few parameters can underfit
- Too many parameters can increase noise sensitivity and trainability issues

Audience question: where is nonlinearity?

Quantum gates are linear operations on amplitudes.

Nonlinearity in learning appears through:

- nonlinear dependence of probabilities on amplitudes
- nonlinear loss definitions on measured statistics
- classical post-processing in hybrid loops

Cost functions are expectation values

Expectation-value objective

Many variational objectives are expectations of observables:

$$C(\theta) = \langle O \rangle_{\psi(\theta)} = \langle \psi(\theta) | \, O \, | \psi(\theta) \rangle \, .$$

If O is a sum of Pauli strings,

$$O = \sum_j c_j P_j, \qquad C(\theta) = \sum_j c_j \langle P_j \rangle \, .$$

Expectation values from measurement outcomes

For a single-qubit Pauli observable, measurements produce outcomes $0/1$.

Example with Z:

$$\langle Z \rangle = P(0) - P(1) = 1 - 2P(1).$$

For a general Pauli string P, we first rotate into the P-eigenbasis (basis change), then measure with computational-basis sampling.

$$\langle P \rangle = \sum_{b \in \{0,1\}^n} (-1)^{\mathrm{parity}(b)} P(b).$$

Practical point: finite shots make $P(b)$ a statistical estimate.

One-qubit worked objective

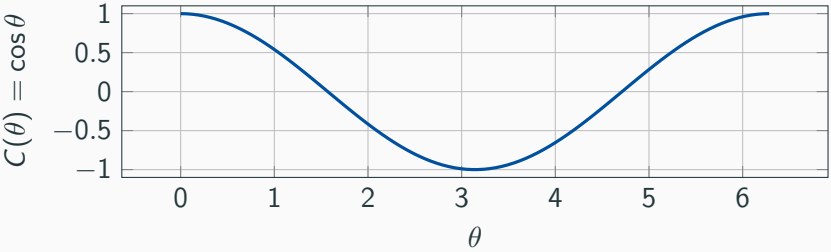
Let $|\psi(\theta)\rangle = R_y(\theta)\,|0\rangle$ and $O = Z$.

$$R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle\,.$$

$$C(\theta) = \langle Z\rangle = \cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2} = \cos(\theta).$$

`lecture_06_demo.ipynb`, Demo 2: one-qubit objective curve.

Geometry of a simple objective



`lecture_06_demo.ipynb`, Demo 2: generated with Python.

VQE as a prototype

Variational principle (energy minimization)

Given a Hamiltonian H, define

$$E(\theta) = \langle H \rangle_{\psi(\theta)} .$$

For any trial state from the ansatz:

$$E(\theta) \geq E_0$$

where E_0 is the true ground-state energy.

VQE loop

1. Choose ansatz and initialize parameters θ
2. Estimate $E(\theta)$ from circuit evaluations
3. Update θ with a classical optimizer
4. Repeat until stopping criterion is met

This is a hybrid quantum-classical optimization loop.

VQE loop as two connected modules

The key learning signal is the *objective value* (an expectation value), not the full statevector.

See also: Lecture 7 will explain how gradients can be computed from circuits.

Toy VQE example

Let $H = Z$, ansatz $|\psi(\theta)\rangle = R_y(\theta)\,|0\rangle$.

$$E(\theta) = \langle Z \rangle = \cos(\theta).$$

The minimum occurs at $\theta = \pi$ with

$$E_{\text{min}} = -1.$$

`lecture_06_demo.ipynb`, Demo 4: numerical verification.

Two-qubit objective as a Pauli sum

Example Hamiltonian:

$$H = Z \otimes Z + \frac{1}{2}(X \otimes I + I \otimes X).$$

This is measured as a weighted sum of Pauli expectations.

- natural fit for an expectation-value estimator
- useful toy template for chemistry-style Hamiltonians

`lecture_06_demo.ipynb`, Demo 3 and Demo 5: Qiskit implementation.

Which measurement settings do we need?

In principle, a Pauli-sum Hamiltonian uses multiple measurement bases. Consider

$$H = Z \otimes Z + \frac{1}{2}(X \otimes I + I \otimes X).$$

Term	Pauli basis	What we measure
$Z \otimes Z$	Z on both qubits	correlator $\langle Z_0 Z_1 \rangle$
$X \otimes I$	X on qubit 0	$\langle X_0 \rangle$
$I \otimes X$	X on qubit 1	$\langle X_1 \rangle$

Practical note: commuting terms can be grouped to reduce settings.

This grouping idea will be revisited when discussing shot budgets.

Practical optimization behavior

Optimizer families in VQAs

Family	Examples	Typical use case
Gradient-free	COBYLA, Nelder-Mead	small-medium parameter counts
Stochastic	SPSA	noisy objectives, many parameters
Gradient-based	Adam, L-BFGS	when reliable gradients are available

No single optimizer is universally best.

Shot noise and convergence

With finite shots, objective evaluations are noisy estimates.

Consequences:

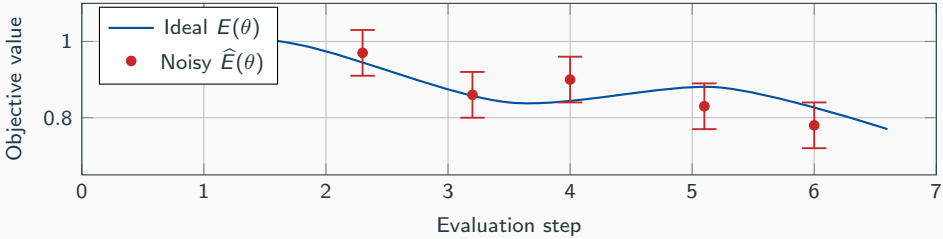
- run-to-run variability in final parameters
- occasional false improvements or regressions
- need for restart checks and robust logging

Schematic view of shot noise

Think of the ideal objective as $E(\theta)$, but you observe

$$\widehat{E}(\theta) = E(\theta) + \epsilon,$$

where ϵ is a random error coming from sampling.



More shots reduce the spread, but increase cost.

Same objective, different trajectories

Even on simulators, optimizer updates can follow different paths and settle at different local minima.

`lecture_06_demo.ipynb`, Demo 5: optimizer comparison plot.

Implementation checklist

Qiskit and PennyLane workflow

Qiskit 2.x:

- use `StatevectorEstimator` for expectation-value objectives
- use sampler primitives when explicit count statistics are needed

PennyLane:

- define a QNode returning `qml.expval(...)`
- optimize parameters with a classical optimizer wrapper

`lecture_06_demo.ipynb`, Demo 6: one-qubit cross-check.

Summary

- PQCs define trainable quantum-state families
- objectives in VQAs are expectation values of observables
- VQE is the canonical variational optimization template
- optimizer choice and shot budget are practical design variables

Further reading and references

- PennyLane VQE demo: https://pennylane.ai/qml/demos/tutorial_vqe/
- Video (example inspiration): https://www.youtube.com/watch?v=DUq-0r-Prw0
- Video (example inspiration): https://www.youtube.com/watch?v=qiRtUUZ5s9s

These are included for implementation perspective and visual intuition. The lecture math and loop structure remain framework-agnostic.

Forward pointer to Lecture 7

We now have variational objectives and optimization loops.

Next question: how do we compute gradients efficiently and reliably?

Lecture 7 introduces:

- parameter-shift rule
- hybrid chain-rule backpropagation
- SPSA and gradient-cost trade-offs